3. Project Design Phase

FitBuddy – AI Fitness Plan Generator using Gemini Models

Phase Report

Prepared from the supplied FitBuddy project documentation and organized into a project lifecycle deliverable.

1. Architecture Overview

FitBuddy uses a modular FastAPI architecture. The frontend uses HTML and Jinja2 templates. Route handlers validate form inputs and coordinate the AI and database layers. Gemini models generate workout plans and tips. SQLAlchemy persists records in SQLite.

2. Main Components

Frontend templates: index.html, result.html, all_users.html. Backend: main.py and routes.py. AI layer: workout generation, nutrition-tip generation, and feedback-based plan updating. Database layer: SQLAlchemy models and CRUD helpers. Static assets: CSS and optional images.

3. Request Flow — Generate Plan

User opens the home page → submits details → FastAPI validates fields → workout generator requests a structured 7-day plan → nutrition generator creates a concise tip → database saves user and plan → result template displays the response.

4. Request Flow — Feedback

User submits user ID and feedback → backend retrieves the stored plan → Gemini receives the original plan and feedback → updated plan is saved separately → result page displays the revised plan.

5. Data Design

User record fields: database ID, user ID, username, age, weight, goal, intensity, creation timestamp. Plan record fields: plan ID, user reference, original workout plan, updated workout plan, nutrition tip, feedback, and timestamps. A relational link connects each plan to a user.

6. AI Prompt Design

Workout prompts should request a seven-day structured plan with day labels, workout focus, warm-up, exercises, sets/reps or duration, cooldown, and recovery guidance. Nutrition prompts should request a concise, goal-aligned tip. Update prompts should include the original plan and user feedback.

7. Error Handling and Security

Validate age and weight; handle missing user IDs and plans; catch API errors; keep the Gemini API key in .env; exclude .env from version control; do not expose secrets in error messages.

8. Design Outcome

A separation-of-concerns design makes the project easier to test and maintain while supporting future extensions such as authentication, progress tracking, and cloud deployment.

Note: This document describes the intended design and workflow. Test outcomes should be recorded only after the application has been run and verified.